



华南师范大学

本科学生实验（实践）报告

鳕鱼音乐生成软件

项目计划书

院系：计算机学院
实验课程：web项目开发
开课时间：2026.3.7
专业：网络工程

项目成员：汪炜力 杨茵棋

提交日期：2026年3月20日

修订记录

版本	修订日期	修订内容	作者
V0.1	2026-03-12	初稿完成，确定技术方案	汪炜力
V0.2	2026-03-18	补充进度计划与风险管理	汪炜力
V1.0	2026-03-20	正式发布，评审通过	汪炜力

目录

1. 引言
2. 项目概述
3. 项目组织与分工
4. 技术方案
5. 进度计划
6. 资源计划
7. 质量保证计划
8. 风险管理
9. 配置与变更管理
10. 沟通管理
11. 项目验收与交付
12. 附录

一、引言

1.1 编写目的

本计划书是鳕鱼音乐生成软件项目的核心指导文件，具体用途如下：

作为项目实施的指导文件，统一团队成员对项目目标、范围、技术方案的理解，确保开发工作有据可依；作为项目进度、质量与资源的管控依据，便于识别偏差并及时纠偏；作为项目验收与成绩评估的标准文件，供课程指导教师审阅与评分参考；为项目相关方（指导教师、组员）提供决策参考，保障项目按时、按质交付。

1.2 项目背景

随着人工智能生成内容（AIGC）技术的快速发展，AI辅助音乐创作已成为前沿应用方向。当前市场上面向普通用户的在线音乐生成工具较少，操作门槛较高，缺乏简洁易用的Web端产品。

本项目为两人大学生Web开发大作业，旨在结合前后端开发技术与AI音乐生成API，构建一款面向普通用户的在线音乐生成平台——鳕鱼音乐生成软件（Codfish Music Generator，简称CMG），让用户通过简单的文字描述或参数设置即可生成个性化音乐片段。

1.3 术语与缩略语

术语/缩略语	说明
CMG	Codfish Music Generator，鳕鱼音乐生成软件项目简称
AIGC	Artificial Intelligence Generated Content，人工智能生成内容
API	Application Programming Interface，应用程序接口
REST	Representational State Transfer，一种Web服务架构风格
SPA	Single Page Application，单页面应用
JWT	JSON Web Token，用于用户认证的令牌机制
ORM	Object Relational Mapping，对象关系映射
CI/CD	Continuous Integration / Continuous Deployment，持续集成/持续部署
UI	User Interface，用户界面
DB	Database，数据库

1.4 参考资料

1. 《软件项目管理》，清华大学出版社，2022年
2. GB/T 8567-2006，《计算机软件文档编制规范》，中国国家标准化管理委员会
3. GB/T 19001-2016，《质量管理体系 要求》
4. Vue.js 官方文档，<https://vuejs.org/guide/introduction.html>
5. FastAPI 官方文档，<https://fastapi.tiangolo.com>
6. Suno AI API 文档（音乐生成服务）
7. MySQL 8.0 官方文档，<https://dev.mysql.com/doc/>
8. Git 官方文档，<https://git-scm.com/doc>

二、项目概述

2.1 项目名称

鳕鱼音乐生成软件（Codfish Music Generator，CMG）

2.2 项目目标

功能目标：

用户可通过自然语言文字描述（如"轻松的爵士风格，适合下午咖啡时光"）生成音乐片段；用户可对生成参数（风格、情绪、时长、速度）进行可视化调节；平台提供音乐播放、下载、历史记录查看等基础功能；用户可注册登录，保存个人生成历史。

性能目标：

系统在两台开发机本地部署环境下，接口平均响应时间（不含音乐生成的AI外部调用时间）不超过500ms；音乐生成任务采用异步轮询机制，生成状态实时反馈至前端；系统支持至少5个并发用户在演示环境下正常使用；前端页面首屏加载时间（本地网络）不超过2秒。

2.3 项目范围

本项目包含：

面向浏览器的Web前端页面（PC端适配）；基于REST API的后端服务；用户注册、登录与鉴权模块；音乐生成任务调度与状态管理；音乐播放与下载功能；个人历史记录管理；对接第三方AI音乐生成API（Suno AI或同类服务）。

本项目不包含：

移动端App开发；多语言国际化；商业化支付功能；社交分享与社区功能；自研AI音乐模型训练；高并发生产级部署与运维。

2.4 项目假设与约束条件

假设条件：

第三方AI音乐生成API服务在开发与演示期间保持可用；两名组员全程参与，无人员变动；指导教师在需求阶段提供一次确认反馈。

约束条件：

团队规模为2人，总可用工时有限；项目须在学期规定截止日期前完成并提交；所有开发工作在个人电脑上进行，不依赖额外付费云服务器；技术栈须以团队已掌握技能为主，不引入完全陌生的框架。

三、项目组织与分工

3.1 组织结构

鳕鱼音乐生成软件项目团队

└─ 汪炜力（项目负责人 / 后端主导）

└─ 杨茵棋（前端主导 / 测试负责人）

3.2 角色与职责

角色	担任人	主要职责
项目负责人	汪炜力	进度协调、文档编写、汇报
前端工程师	杨茵棋	页面开发、接口调用、前后端联调
后端工程师	汪炜力	接口开发、数据库设计、AI 服务集成
测试工程师	杨茵棋	测试用例编写、功能测试、缺陷跟踪

3.3 人员配置

成员	每周可投入工时	参与阶段
汪炜力	约 12 小时	全程
杨茵棋	约 12 小时	全程

四、技术方案

4.1 系统架构设计

系统采用前后端分离架构，分为三层：

表现层（前端）： 基于 Vue 3 构建 SPA，使用 Element Plus 组件库与 Axios 完成页面交互和接口通信。

服务层（后端）： 基于 **Spring Boot** 提供 **RESTful API**，处理业务逻辑、用户鉴权与音乐生成任务调度，通过 **HTTP** 调用第三方 **AI** 服务。

数据层： 使用 **MySQL** 存储用户信息与任务记录；开发阶段可用 **SQLite** 简化环境；音频文件存储于后端本地静态目录。

4.2 技术选型

层级	技术/工具	用途
前端	Vue 3	界面开发
前端 UI	Element Plus	组件库
前端通信	Axios	HTTP 请求
后端	Spring Boot	业务逻辑开发
接口规范	RESTful API	接口设计
数据库	MySQL / SQLite	数据存储
构建工具	Maven	项目构建与管理
认证	JWT	用户鉴权
AI 服务	Suno AI API	音乐生成
版本控制	Git + GitHub	代码管理

4.3 开发方法

本项目采用迭代式开发方法，结合轻量级敏捷实践，共划分 4 个迭代周期。每周进行一次团队内部同步会议，维护任务清单与每周进度记录，确保任务可追溯。

4.4 关键技术难点

难点一：Spring Boot 的学习与上手。 后端成员具备 **Java** 基础，但 **IoC** 容器、注解驱动开发、**JPA** 操作数据库等内容在课程中未系统涉及，存在学习曲线，已在计划前期预留专项学习时间。

难点二：Vue 3 Composition API 的学习。 前端成员熟悉传统 **JavaScript**，但 **Vue 3** 的组合式 **API** 是全新概念，需要专项学习时间，同样已在前期计划中安排。

难点三：前后端跨域配置。 前后端开发服务器端口不同存在 **CORS** 问题，需在后端统一配置跨域策略，并通过 **Vite** 代理解决本地开发跨域。

难点四：**AI 生成接口的异步处理**。AI 服务生成音乐耗时较长，需实现异步任务提交与状态轮询机制，前端需展示友好的等待状态反馈。

五、进度计划

5.1 阶段划分

阶段一：需求分析与技术预研（第 1-4 周） 完成需求文档、UI 原型、数据库表结构设计与接口协议文档初稿；同步完成 Vue 3 与 Spring Boot 基础学习，搭建本地开发环境并完成 Hello World 验证。

阶段二：基础框架搭建（第 5-6 周） 搭建前后端项目脚手架；实现用户注册、登录接口与 JWT 鉴权，完成前后端注册登录流程联通。

阶段三：核心功能开发（第 7-11 周） 实现音乐生成参数页面；完成 AI 接口对接与异步任务调度；实现音乐在线播放与文件下载；完成历史记录列表与查询功能。

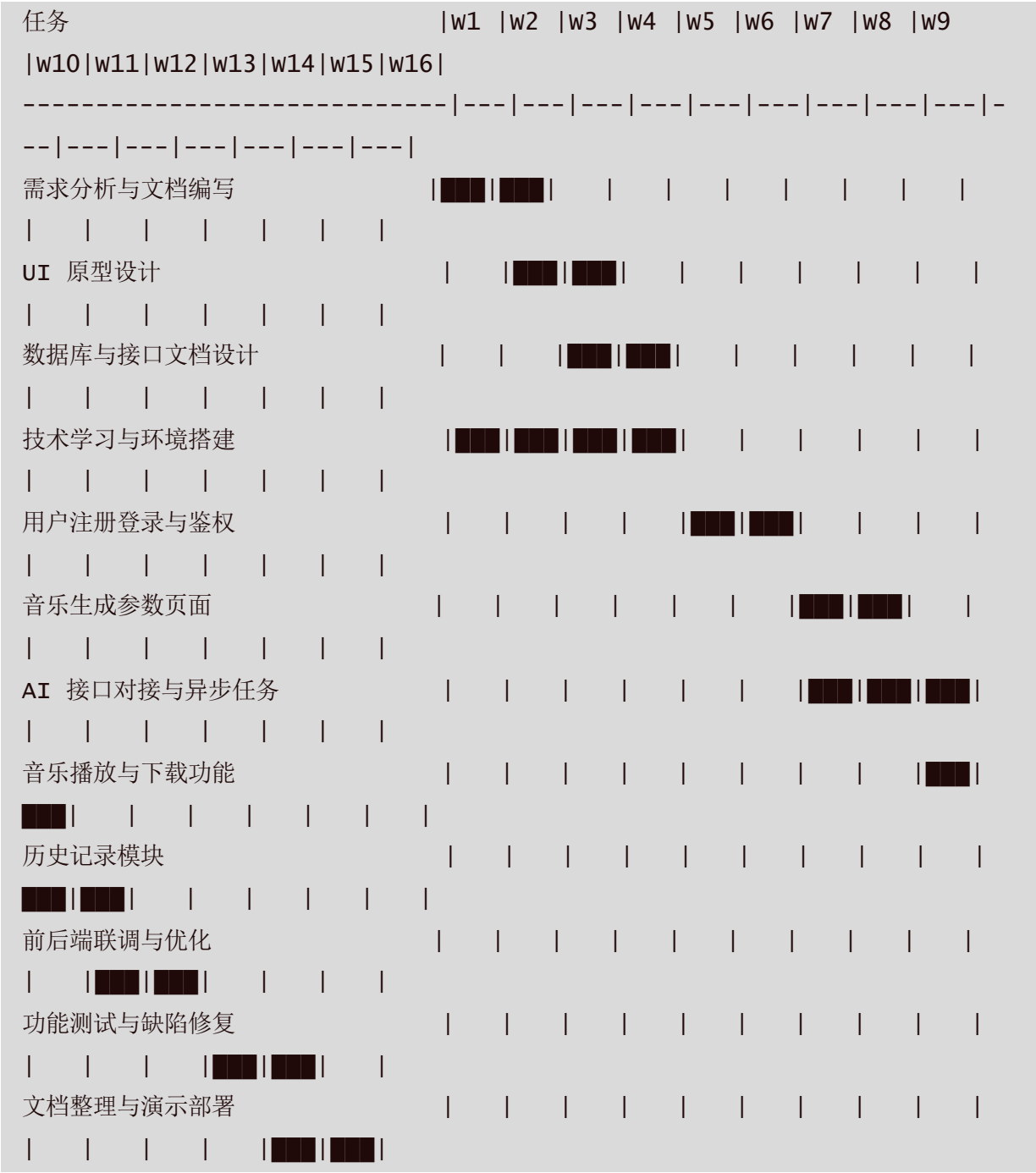
阶段四：联调与优化（第 12-13 周） 前后端完整联调，修复接口数据不一致问题；完善错误提示与边界处理；完成 Chrome、Edge 兼容性验证。

阶段五：测试、文档与交付（第 14-16 周） 执行完整功能测试，修复缺陷；整理项目文档；完成本地演示部署，制作答辩材料。

5.2 里程碑

阶段	目标产出	计划完成时间
需求与技术基线	需求文档、UI 原型、接口协议、开发环境就绪	第 4 周末
框架联通	注册登录前后端可运行，鉴权通过验证	第 6 周末
核心功能完成	音乐生成、播放下载、历史记录可用	第 11 周末
联调完成	前后端完整联通，主流程无阻塞性错误	第 13 周末
项目交付	测试通过、文档齐全、演示环境就绪	第 16 周末

5.3 甘特图



六、资源计划

6.1 人力资源

成员	总投入工时（预估）
汪炜力	约 192 小时（16 周 × 12h）
杨茵棋	约 192 小时（16 周 × 12h）
合计	约 384 小时

6.2 硬件与软件资源

硬件： 两名成员各使用个人笔记本电脑进行开发，演示时使用其中一台作为本地服务器。

软件： 操作系统 Windows 11 / macOS；IntelliJ IDEA 社区版（后端）、VS Code（前端）；Git + GitHub；MySQL Workbench；Postman；Figma；运行环境 JDK 17、Node.js 20.x、Maven 3.9.x、MySQL 8.0，均为免费或开源工具。

6.3 经费预算

支出项目	单价	数量	小计
服务器部署	约 ¥55/年	1 个	¥55

七、质量保证计划

7.1 质量目标

需求文档中明确的全部功能点须通过验收测试，功能符合率达 100%；所有 API 端点须覆盖正常场景与异常场景测试；页面兼容 Chrome、Edge 两款主流浏览器；及时修复以下两种缺陷 P0 级缺陷（系统崩溃或主流程阻塞），P1 级缺陷（功能错误）。

7.2 质量控制措施

前端进行代码风格检查；后端提交前执行 `mvn clean package` 确认无编译错误；采用 GitHub Pull Request 机制，功能分支合入主干前须经另一成员代码审阅；后端集成 SpringDoc OpenAPI，接口变更后文档自动更新，双方以 Swagger 文档为唯一接口协议依据。

7.3 评审与测试策略

在需求基线、核心功能完成、项目交付三个节点分别进行阶段评审。单元测试由开发人员在编码阶段自行完成，覆盖核心业务逻辑；功能测试在第 14-15 周集中执行，依据《功能测试用例清单》逐项验证；集成测试在联调阶段持续进行；兼容性测试在第 15 周末完成。

7.4 验收标准

全部核心功能可正常使用；《功能测试用例清单》全部通过，无未修复的 P0/P1 缺陷；代码已推送至 GitHub，README 含本地运行说明；文档已完整归档；演示答辩时本地环境可完整展示主流程，无崩溃性错误。

八、风险管理

8.1 风险识别与分析

编号	风险描述	发生概率	影响程度	风险等级
R1	学习 Spring Boot 所需时间超出预期，后端开发工时压缩	高	高	高
R2	学习 Vue 3 Composition API 所需时间超出预期，前端进度滞后	高	中	高
R3	Maven 依赖冲突或版本不兼容，环境搭建耗时过长	中	中	中
R4	前后端接口协议不一致，联调阶段大量返工	中	中	中
R5	AI 音乐生成 API 服务不稳定或调用超出免费额度	中	高	高
R6	异步任务机制理解不足，导致该模块开发周期延长	中	高	高
R7	课业集中期工时严重不足，阶段任务无法按时完成	中	中	中
R8	演示环境运行异常，影响答辩现场效果	低	高	中

8.2 风险应对措施

- R1（Spring Boot 学习超期）：** 前期计划中已预留 3 周学习时间，以"能独立完成用户注册接口并调通数据库"为完成标准；若仍超期，优先裁减历史记录导出等非核心功能，保障主流程按时交付。
- R2（Vue 3 学习超期）：** 同样预留学习时间，以"能完成带 Axios 调用的登录页面"为标准；若进度滞后，临时降级为 Options API 实现，待核心功能稳定后再重构。
- R3（Maven 依赖冲突）：** 统一使用 Spring Initializr 生成项目骨架，避免手动引入版本不明的依赖；遇到冲突使用 `mvn dependency:tree` 排查解决。
- R4（接口协议不一致）：** 开发启动前双方共同冻结接口文档，后续变更须通过 GitHub Issue 记录并通知对方，不允许单方面静默修改。
- R5（AI 接口不稳定）：** 开发阶段使用 Mock 接口替代真实 AI 调用；提前调研备用 AI 服务，一旦主服务不可用可在 1 天内完成切换。
- R6（异步任务开发超期）：** 若第 10 周仍未完成异步方案，降级为同步阻塞方案保证功能可用，答辩时说明技术取舍原因。

R7（课业冲突工时不足）：每周初根据课业情况调整任务优先级，将 UI 样式优化、批量操作等非核心功能列为可裁减项。

R8（演示环境异常）：第 14 周起提前完成演示部署验证，留出排查时间；答辩前录制完整演示录屏备用。

九、配置与变更管理

9.1 配置管理工具

本项目使用 Git 进行代码版本控制，托管于 GitHub 私有仓库，仓库命名为 **codfish-music-generator**，目录结构如下：

```
codfish-music-generator/  
├─ frontend/      # vue 3 前端项目  
├─ backend/       # Spring Boot 后端项目  
├─ docs/          # 项目文档  
├─ sql/  
│   └─ init.sql   # 数据库初始化脚本  
└─ README.md
```

9.2 版本控制策略

采用 Git Feature Branch 工作流：**main** 分支为稳定主干，不允许直接提交代码；**dev** 分支为集成分支，功能自测通过后先合入 **dev**，联调验证无误后再合入 **main**；功能分支命名为 **feature/功能名**，缺陷修复分支命名为 **fix/问题名**；每个阶段末在 **main** 分支打版本 Tag。

9.3 变更流程

变更发起人通过 GitHub Issue 描述变更内容与进度影响；双方在周会中评审决定是否接受；接受后同步更新需求文档与接口文档，并在本计划书修订记录中追加记录；若影响阶段节点，同步调整进度计划并通知指导教师。

十、沟通管理

10.1 沟通对象与方式

沟通对象	沟通事项	沟通方式	频率
团队内部	进度同步、技术讨论、任务分配	微信 / 线下面谈	每周至少 1 次
指导教师	阶段汇报、需求确认、答辩准备	邮件 / 课后面谈	阶段节点约 3 次

10.2 汇报机制

每周固定召开一次内部周会，回顾本周完成情况，分配下周任务，确认风险与开发问题，会后在 **GitHub Issues** 中记录简要纪要。遇到阻塞性问题时通过微信即时沟通，不等待周会，确保问题在 1 个工作日内得到响应。

十一、项目验收与交付

11.1 验收流程

第一步：依据《功能测试用例清单》完成内部完整功能测试，整理缺陷清单并修复。

第二步：汇总全部交付物，逐项核对清单，确认文档完整、代码可运行、演示环境就绪。

第三步：向指导教师提交完整交付物，进行演示答辩，记录反馈意见。

第四步：根据反馈在规定时间内完成最终修订，提交最终版本。

11.2 验收标准

全部核心功能（注册登录、音乐生成、播放下载、历史记录）可正常使用；《功能测试用例清单》全部通过，无 P0/P1 未修复缺陷；代码已推送至 **GitHub** 且可按 **README** 本地运行；文档完整归档；演示时本地环境可完整展示主流程，无崩溃性错误。

11.3 交付物清单

交付物	说明
源代码	完整代码，含前端、后端
数据库初始化脚本	<code>sql/init.sql</code> ，含建表语句与初始数据
项目计划书	本文档，V1.0 版本
接口文档	Swagger 导出的接口说明文档
功能测试用例清单	含测试用例与执行结果
用户手册	用户使用说明书
README	项目介绍与本地运行步骤
演示录屏（备用）	完整主流程操作录屏视频
答辩 PPT	项目答辩演示材料